Household members

This document describes how parent–dependent households, login, and membership application approval work in FundFlow.

Core rules (enforced in code)

Situation	Behavior
Each member	Gets their **own `User`** (unique login; no shared `user_id`)
Same email in CSV import	First row = parent application; later rows with same `household_email` = dependent applications
Different email, same family	Admin links via **Parent member** (applications use `parent_application_id` only when import shares email; otherwise admin assigns after approval)
Separated dependent (`email` household email)	`is_separated` + `direct_login_enabled`; `User.email` = personal email
Household-only dependent (same contact email)	`User.email` = internal address (`*@household.members.local`); login via household email + profile picker + their password
Applicants	Cannot choose a parent (only admins via member form / `parent_application_id` on import)

Main components

Piece	Role
`HouseholdMemberService`	Create from application/admin, assign/remove parent, sync flags, validate cycles and blocked parents
`MembershipApplicationApprovalService`	Always creates one user per member with correct household flags
`MemberUserEmail`	Resolves unique user emails; uses internal addresses when the household email is already taken
`User::activeMember()` + `session('active_member_id')`	Profile picker logs in as the selected member's user so portal data stays correct
`CurrentMember`	Member panel reads the active member consistently
Admin UI	Parent dropdown (household heads only); household/separated toggles are read-only and computed

Login paths

1. **Household login** — User enters the parent's `household_email` and password, picks a profile, then verifies with that profile's password or PIN.
2. **Direct login** — Separated dependents (`direct_login_enabled`) sign in with their own `User.email` and password; no profile picker.

Import behavior

- Rows are processed **in file order**.
- Grouping uses `household_email` when present, otherwise the row `email`.
- The **first** row in a group becomes the parent application (`parent_application_id` is null).
- **Later** rows in the same group get `parent_application_id` pointing at the first application.
- Rows with a **different email** are **not** auto-linked; an administrator must assign **Parent member** after approval.

Approval behavior

- **Parent application** — Creates a `User` and parent `Member` with `household_email` set to the household login email.
- **Dependent application** — Creates a separate `User` and dependent `Member` with `parent_member_id` set.
- Bulk approve orders parents before dependents; approving a dependent can auto-approve a pending parent first.
- **Same contact email as household** — Dependent gets an internal login email; not separated.
- **Different contact email** — `is_separated` and `direct_login_enabled` are set; `User.email` is the personal email.

Admin: add member to family later

1. Create or locate the member (with their own email and portal password).
2. Set **Parent member** to the household head (applicants cannot do this).
3. The system sets `household_email`, `is_separated`, and `direct_login_enabled` from the parent and contact email.

Edge cases

Case	Behavior
Parent not active	Cannot assign new dependents to that parent
Parent suspended / withdrawn	Household login may still reach the picker; only members with **active** status can open the portal. Separated dependents can use **direct login** if their own membership is active
Removing parent	Member becomes an independent household head with their own email
Email changes	`HouseholdAccessService` handles separating from or rejoining the household
Circular parent assignment	Rejected (member cannot be parent of their dependent, etc.)

Database

Tenant migration adds to `membership_applications`:

- `household_email`
- `parent_application_id`
- `member_id` (created member after approval)

Run on each tenant:

```
php artisan tenants:migrate
```

Existing data

Members approved **before** this change may still share one `user_id` or lack `parent_member_id`. Fix by:

1. Re-importing applications (after removing incorrect pending/approved records), or
2. Manually setting **Parent member** and ensuring each member has a distinct `User` in admin.

Tests

- `tests/Feature/Tenant/HouseholdMemberProvisioningTest.php`
- `tests/Feature/Tenant/MembershipApplicationTest.php` (household import/approve cases)
- `tests/Feature/Tenant/HouseholdProfileManagementTest.php`

```
php artisan test --compact tests/Feature/Tenant/HouseholdMemberProvisioningTest.php tests/Feature/Tenan
```

Related code paths

- `app/Services/Tenant/HouseholdMemberService.php`
- `app/Services/MembershipApplicationApprovalService.php`
- `app/Services/MembershipApplicationImportService.php`
- `app/Services/Tenant/HouseholdAccessService.php`
- `app/Support/MemberUserEmail.php`
- `app/Support/Tenant/CurrentMember.php`
- `app/Livewire/Tenant/MemberLoginPage.php`
- `app/Filament/Tenant/Resources/Members/Schemas/MemberForm.php`